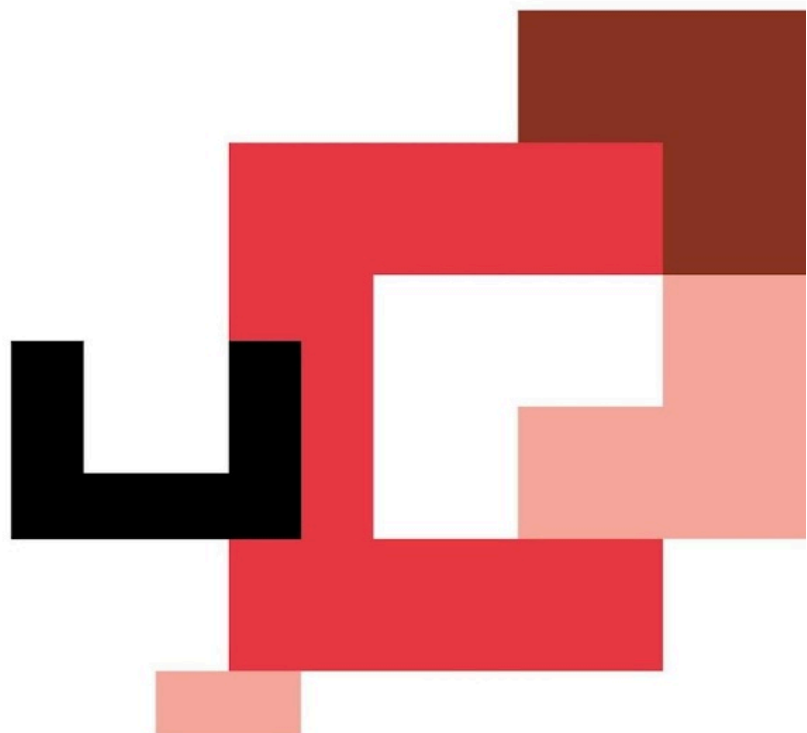




FULL STACK DEVELOPER + IA

Módulo 2 | Feature 1

Montando la estructura



Objetivo de la feature

Construir tu **primer backend** con **Node.js + Express**, entendiendo:

- Qué es un servidor y cómo **escucha** peticiones.
- Qué es una **API** y cómo se diseñan **rutasyendpoints**.
- Cómo responder **siempre en JSON** con un formato estándar.

Requisitos

Tech / configuración

- **Node.js 18+** (última versión LTS)
- Proyecto con **ES Modules**
 - En `package.json`: `"type": "module"`
- Script de ejecución con recarga:
 - `"start": "node --watch src/server.js"`

Estructura obligatoria

```
None

src/

├─ db/

|   └─ products.js      # Datos de ejemplo (mock data)

├─ routes/

|   └─ index.routes.js  # Punto de entrada de rutas

|   └─ products.routes.js # Rutas específicas para productos

├─ app.js              # Configuración de la aplicación
Express

└─ server.js           # Punto de inicio del servidor
```

Endpoints obligatorios

Método	Ruta	Descripción
GET	<code>/health</code>	Estado del servidor
GET	<code>/api/products</code>	Listar todos los productos
GET	<code>/api/products/:id</code>	Obtener producto por ID
GET		404 JSON

Respuestas estándar (obligatorio)

Éxito

JSON

```
{ "ok": true, "data": { "...": "..." } }
```

Error

JSON

```
{ "ok": false, "error": { "message": "..." } }
```

Qué hay que hacer (paso a paso)

1) Inicializa el proyecto

1. Crea carpeta del sprint y ejecuta:
 - `npm init -y`
2. Instala Express:
 - `npm i express`
3. Ajusta `package.json`:

- `"type": "module"`
 - `"start": "node --watch src/server.js"`
-

2) Crea `src/app.js` (config de Express)

En `app.js` debes:

- Crear `app = express()`
- Activar JSON:
 - `app.use(express.json())`
 - `app.use(express.urlencoded({extended: true}))`
- Conectar las rutas desde `/routes`:
 - `app.use(...)`

Objetivo: `app.js` define la app; **no debe arrancar el servidor.**

3) Crea `src/server.js` (arranque del servidor)

En `server.js` debes:

- Importar `app` desde `app.js`
- Hacer `app.listen(PORT, ...)`
- Usar `PORT = 3000`

Objetivo: `server.js` solo “enciende” el servidor.

4) Crea rutas

`src/routes/index.routes.js`

Debe incluir:

- `GET /health` → devuelve `{ ok:true, data: { status: "up" } }` (o similar)

`src/routes/products.routes.js`

Debe incluir:

- `GET /api/products` → devuelve array de productos (hardcoded)
- `GET /api/products/:id` → devuelve 1 producto por ID

- Si no existe: **404** con `{ ok:false, error:{ message:"Product not found" } }`
-

5) 404 global en JSON

Debe existir un manejador final para rutas inexistentes:

- **GET** → **404 JSON** estándar
-

Uso de la IA en este sprint

Para guiarte (sin que te escriba el sprint entero)

- Pedir explicación de conceptos (servidor, rutas, req/res, params).
- Pedir revisión de estructura y consistencia del JSON.

Prompts útiles (copia/pega):

1. Diseño de respuestas

“Estoy haciendo una API en Express. Quiero que todas las respuestas sigan el formato `{ ok:true, data }` y los errores `{ ok:false, error:{ message } }`. ¿Cómo organizo mis rutas para no repetir código y mantenerlo simple en un sprint inicial?”

2. Revisión de endpoints

“Revisa si mis endpoints cumplen: GET /health, GET /api/products, GET /api/products/:id y 404 JSON. Dime qué casos me faltan (id inexistente, id inválido) y cómo responder correctamente.”

3. Debug (si tenemos un error)

“Tengo este error al levantar Express / al acceder a una ruta. Te paso el mensaje y el archivo implicado. Dime las 3 causas más probables y cómo comprobar cada una.”

Regla: no pegues código de IA sin entenderlo. Si lo usas, coméntalo en tu README: “Qué me sugirió la IA y qué ajusté yo”.

Pistas

- **Separación de responsabilidades**
 - `server.js` enciende el servidor.
 - `app.js` configura middlewares y conecta rutas.
 - **Params**
 - `:id` llega en `req.params.id` (string). Normalmente conviertes a número si tus IDs son numéricos.
 - **Orden importa**
 - El **404** va al final, después de conectar todas las rutas.
-

Importante que tienen que tener en cuenta

1. Todo es JSON

No devuelvas HTML, no renderices vistas, no uses templates.

2. Formato estándar

Mantén el formato `{ok,data}` / `{ok:false,error}` incluso en 404.

3. Errores claros

Si el producto no existe, la respuesta debe ser **404** (no 200 con error dentro).

4. Endpoints exactos

`/api/products` y `/api/products/:id` deben coincidir exactamente con la tabla.

5. Recarga con `--watch`

Si no recarga: revisa versión de Node (18+) y el script.

Checks rápidos (autoevaluación)

- `npm start` levanta el servidor sin errores
 - `GET /health` → `{ ok:true, data: ... }`
 - `GET /api/products` → `{ ok:true, data: [ ... ] }`
 - `GET /api/products/1` → `{ ok:true, data: { ... } }`
 - `GET /api/products/999` → **404** `{ ok:false, error:{ message } }`
 - Ruta inexistente → **404** en JSON estándar
-

Ejemplos cURL (para entregar en el README)

Health check

Shell

```
curl http://localhost:3000/health
```

Listar productos

Shell

```
curl http://localhost:3000/api/products
```

Obtener producto por ID

Shell

```
curl http://localhost:3000/api/products/1
```

Producto no encontrado (404)

Shell

```
curl http://localhost:3000/api/products/999
```

Ruta inexistente (404)

Shell

```
curl http://localhost:3000/ruta-que-no-existe
```
